

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5115 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I M Nishanth (192511154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M Nishanth**

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Question 1: MD5 Weaknesses in Password Storage and a Secure SHA-Based Alternative

Understanding of the Case

A software company stored user passwords using MD5, and after a breach, attackers were able to crack a large number of them. This indicates that MD5's design characteristics directly enabled the compromise rather than the breach alone being the sole cause.

Analysis of MD5 Weaknesses

1. High computational speed: MD5 was designed to hash data quickly, which is desirable for checksums but undesirable for password storage. Modern GPUs and ASICs can compute billions of MD5 hashes per second, making brute-force and dictionary attacks highly efficient.
2. Absence of built-in salting: If the application applied MD5 directly to passwords without a unique per-user salt, identical passwords produce identical hashes. This allows attackers to use precomputed rainbow tables to reverse hashes almost instantly.
3. Collision vulnerabilities: Wang et al. (2004) demonstrated practical collision attacks against MD5, showing that two different inputs can produce the same digest. While this specifically affects authenticity/integrity use-cases more than password cracking, it confirms MD5 no longer meets modern cryptographic strength requirements.
4. Short digest and outdated design: MD5's 128-bit output offers weaker collision resistance than modern alternatives, and it lacks the memory-hardness needed to resist GPU/ASIC-accelerated cracking.

Proposed Secure Hashing Approach

A defensible password storage design should combine a strong hash function with key-stretching and salting, rather than relying on a single fast hash:

1. Use SHA-256 or SHA-3 as the underlying cryptographic hash rather than MD5, since both offer stronger collision resistance and are not practically broken.
2. Apply a memory-hard, iterated key-derivation function such as Argon2id (preferred), scrypt, or PBKDF2-HMAC-SHA256 with a high iteration count (e.g., 100,000+), rather than a single pass of SHA-256, since a single fast hash is still crackable at scale.

3. Generate a unique, cryptographically random salt (at least 16 bytes) per user and store it alongside the hash, preventing rainbow-table and cross-account attacks.
4. Optionally add a server-side secret 'pepper' stored separately (e.g., in an HSM or environment secret store) to protect against full database exfiltration.
5. Enforce password complexity policies and rate-limit login attempts to slow online brute-force attempts.

This layered approach (SHA-based hash + salt + adaptive work factor) ensures that even if the database is breached again, cracking passwords becomes computationally infeasible at scale.

Question 2: Digital Signature Flaws in a Banking Application

Understanding of the Case

A banking application uses digital signatures to verify transactions, yet unauthorized transactions are being approved. This means the signature scheme is either not being verified correctly, is using a weak algorithm, or the private key material has been compromised.

Analysis of Possible Flaws

1. Weak or outdated signature algorithm: Use of RSA with insufficient key length (e.g., 1024-bit) or hashing with a broken function (MD5/SHA-1) inside the signature scheme makes forgery computationally feasible.
2. Improper padding scheme: Using raw RSA or PKCS#1 v1.5 padding without proper validation can permit signature malleability, allowing an attacker to construct a valid-looking signature without the private key under certain conditions.
3. Missing replay protection: If the signed transaction data does not include a unique nonce, sequence number, and timestamp, a previously valid signed transaction can be resubmitted (replayed) and accepted again.
4. Insecure private key storage: If signing keys are stored in application memory, config files, or a shared server without hardware protection, attackers who breach the server can sign fraudulent transactions directly.
5. Incomplete verification logic: A common implementation flaw is that the server accepts a transaction without fully validating the signature, the signer's certificate, and its chain of trust, or fails to reject unsigned/mismatched-algorithm requests.

Recommended Improvements

1. Adopt modern, standards-based signature schemes: RSA-PSS or ECDSA (NIST P-256/P-384) combined with SHA-256/SHA-384, avoiding deprecated hash functions.
2. Bind each signature to transaction-specific data including a unique nonce, timestamp, and account/session identifiers, and reject any transaction whose signed payload has already been seen (replay detection).
3. Store private signing keys in a Hardware Security Module (HSM) or secure enclave rather than application-level storage.
4. Enforce strict server-side verification: validate the signature, the certificate chain, certificate validity period, and revocation status (via CRL/OCSP) before approving any transaction.
5. Layer digital signatures with multi-factor authentication (e.g., OTP or push approval) for high-value transactions, and implement real-time anomaly monitoring to flag suspicious approvals.

Question 3: Kerberos Replay Attack in an Enterprise Network

Understanding of the Case

An enterprise network using Kerberos for authentication suffered a replay attack, meaning an attacker captured legitimate authentication traffic and resubmitted it to impersonate a valid user without knowing their credentials.

How the Attack Could Occur

1. Kerberos relies on timestamps within authenticators to prove freshness. If the Key Distribution Center (KDC) or application server does not strictly enforce the standard short validity window (typically about 5 minutes) or lacks a replay cache, a captured authenticator or ticket can be resent successfully.
2. Excessive clock skew tolerance between client, KDC, and servers widens the window during which a captured ticket remains 'fresh' and acceptable.
3. Use of weak legacy encryption types (such as DES or RC4) for tickets makes interception and tampering easier, since these ciphers are more susceptible to cryptanalysis than AES.
4. A 'pass-the-ticket' scenario, where an attacker extracts a valid Ticket Granting Ticket (TGT) or service ticket from a compromised host's memory and reuses it on another machine before expiry, without the server detecting duplicate use.

Proposed Enhancements

1. Maintain strict time synchronization (NTP) across all Kerberos participants and enforce a tight authenticator validity window, rejecting authenticators outside the permitted skew.
2. Implement and enforce a replay cache on the KDC and application servers so that any authenticator or ticket that has already been presented is immediately rejected.
3. Migrate from legacy encryption types (DES/RC4) to AES-256 for all ticket encryption to increase cryptographic strength.
4. Reduce ticket lifetimes and enforce renewal policies, limiting the window of opportunity if a ticket is intercepted.
5. Enable Kerberos pre-authentication hardening (e.g., FAST/armoring) and monitor for anomalous patterns such as the same ticket being used from multiple source IP addresses in a short period.

Question 4: Message Integrity Failure and HMAC-Based Solution

Understanding of the Case

A secure communication system relies on a simple (unkeyed) hash function to verify message integrity, but attackers are able to modify messages without detection. This indicates the scheme lacks authentication, not just integrity.

Why a Simple Hash Fails

1. A plain hash such as $H(\text{message})$ is a public, keyless function — anyone, including an attacker, can recompute the hash after altering the message and simply attach the new, correct hash. The receiver has no way to know the hash was regenerated by an attacker rather than the legitimate sender.
2. If the design naively tries to add a secret by computing $H(\text{key} \parallel \text{message})$, many hash constructions (such as MD5, SHA-1, and SHA-256, which use the Merkle–Damgård construction) are vulnerable to length-extension attacks, letting an attacker append data and compute a valid new hash without knowing the secret key.
3. Without a secret component bound cryptographically into the verification value, the mechanism only offers accidental-corruption detection (like a checksum), not protection against a deliberate, intelligent adversary.

Proposed Solution: HMAC

HMAC (Hash-based Message Authentication Code) resolves this by combining the message with a shared secret key through a nested hashing construction:

1. HMAC is computed as $H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel \text{message}))$, which structurally defeats length-extension attacks and requires knowledge of the secret key K to produce a valid tag.
2. Recommend HMAC-SHA256 (or HMAC-SHA3) using a securely generated and securely shared/exchanged key between sender and receiver.
3. The sender computes the HMAC over the message and transmits both; the receiver recomputes the HMAC using the shared key and compares it (using a constant-time comparison to avoid timing attacks) — any tampering with the message invalidates the HMAC because the attacker does not possess the key.
4. Combine this with proper key management practices: periodic key rotation, secure key exchange (e.g., via TLS or a key management service), and never reusing keys across unrelated systems.

This ensures both integrity (detecting modification) and authenticity (confirming the message originated from a party possessing the shared key), which a bare hash function cannot provide.

Question 5: X.509 Certificate Trust Validation Errors

Understanding of the Case

An organization implementing X.509 certificates for secure communication is experiencing frequent trust validation errors, pointing to weaknesses in certificate management and chain-of-trust configuration rather than the X.509 standard itself.

Analysis of Possible Issues

1. Expired certificates: Certificates not renewed before their validity period ends will immediately fail validation on the client side.
2. Incomplete certificate chain: If the server only presents its leaf (end-entity) certificate without the required intermediate CA certificate(s), clients cannot build a full path to a trusted root, producing errors such as 'unable to get local issuer certificate.'
3. Untrusted or self-signed certificates: Certificates issued by an internal/private CA or self-signed without that CA's root being installed in client trust stores will be rejected by default.

4. Revocation-check failures: If Certificate Revocation Lists (CRLs) or OCSP responders are unreachable, misconfigured, or the certificate has actually been revoked, clients may fail validation or, worse, silently skip revocation checks.
5. Hostname/SAN mismatch: If the certificate's Common Name or Subject Alternative Name fields do not match the domain being accessed, browsers and clients will reject the connection.
6. Client-side clock skew: Since certificate validity is time-bound, an inaccurate system clock on the client can cause a valid certificate to appear expired or not-yet-valid.

Recommended Solutions

1. Implement automated certificate lifecycle management (e.g., ACME-based issuance/renewal such as Let's Encrypt, or an enterprise PKI management tool) with expiry monitoring and alerts well before expiration.
2. Always deploy the complete certificate chain (leaf plus all intermediate CA certificates) on the server, and validate configuration using standard SSL/TLS testing tools.
3. For internal services, distribute the private root/intermediate CA certificates to all client trust stores through centralized configuration management, or use certificates from a publicly trusted CA where feasible.
4. Enable OCSP stapling to provide efficient, reliable revocation status without depending on client-side reachability of the CA's OCSP responder.
5. Regularly audit SAN/CN fields against actual service domains, enforce NTP time synchronization across all systems, and adopt a formal PKI governance policy covering issuance, renewal, and revocation procedures.

Conclusion

Across all five scenarios, the recurring theme is that legacy or improperly configured cryptographic mechanisms (unsalted fast hashes, weak signature padding, permissive replay windows, unkeyed integrity checks, and mismanaged certificate chains) create exploitable gaps. Aligning each system with current standards — SHA-256/Argon2 for password storage, RSA-PSS/ECDSA with replay-resistant signing for transactions, AES-based Kerberos with replay caching, HMAC for message authentication, and disciplined X.509 lifecycle management — restores integrity, authentication, and non-repudiation as required by CO3.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand